

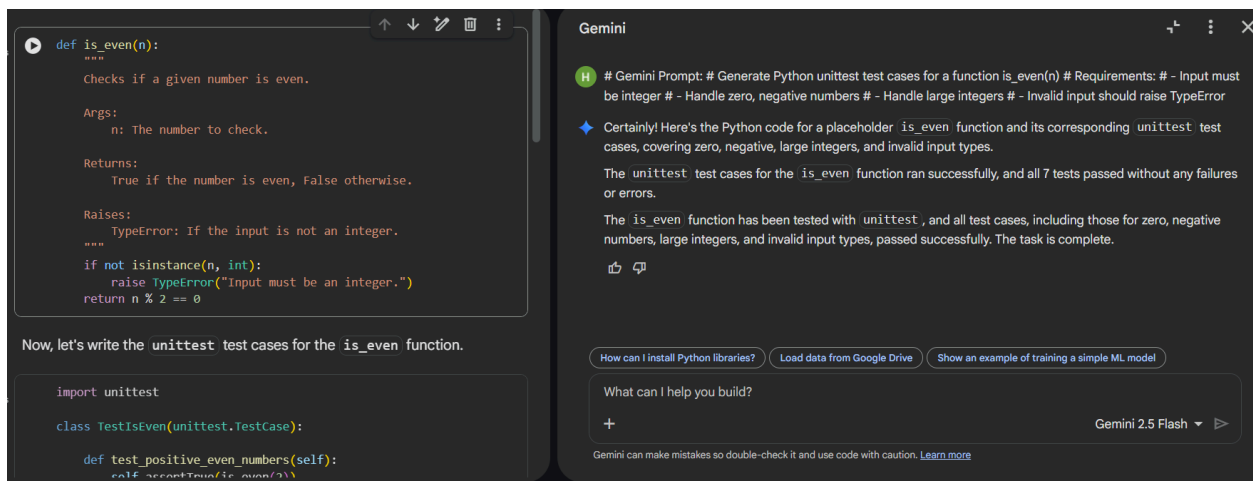
AI Assisted Coding Assignment - 8

Chilukala Sai Architha(2303A52220) - Batch 37

Task1–Test-Driven Development for Even/Odd Number Validator

Use AI tools to first generate test cases for a function `is_even(n)` and then implement the function so that it satisfies all generated tests.

Prompt: Generate Python unittest test cases for a function `is_even(n)` with integer validation, zero, negative, large integers, and invalid input handling.



The screenshot shows a code editor on the left and the Gemini chat interface on the right. The code editor contains the following Python code:

```
def is_even(n):  
    """  
    Checks if a given number is even.  
  
    Args:  
        n: The number to check.  
  
    Returns:  
        True if the number is even, False otherwise.  
  
    Raises:  
        TypeError: If the input is not an integer.  
    """  
    if not isinstance(n, int):  
        raise TypeError("Input must be an integer.")  
    return n % 2 == 0
```

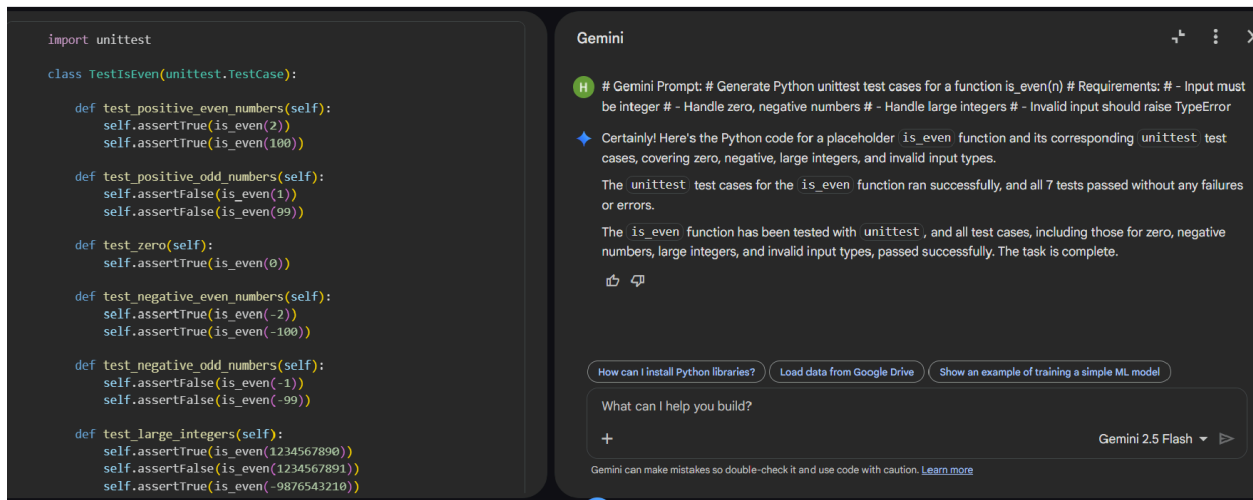
Below the code editor, the text "Now, let's write the unittest test cases for the `is_even` function." is displayed. The chat interface on the right shows the Gemini prompt and response:

Gemini Prompt: # Generate Python unittest test cases for a function `is_even(n)` # Requirements: # - Input must be integer # - Handle zero, negative numbers # - Handle large integers # - Invalid input should raise `TypeError`

Gemini Response: Certainly! Here's the Python code for a placeholder `is_even` function and its corresponding `unittest` test cases, covering zero, negative, large integers, and invalid input types.

The `unittest` test cases for the `is_even` function ran successfully, and all 7 tests passed without any failures or errors.

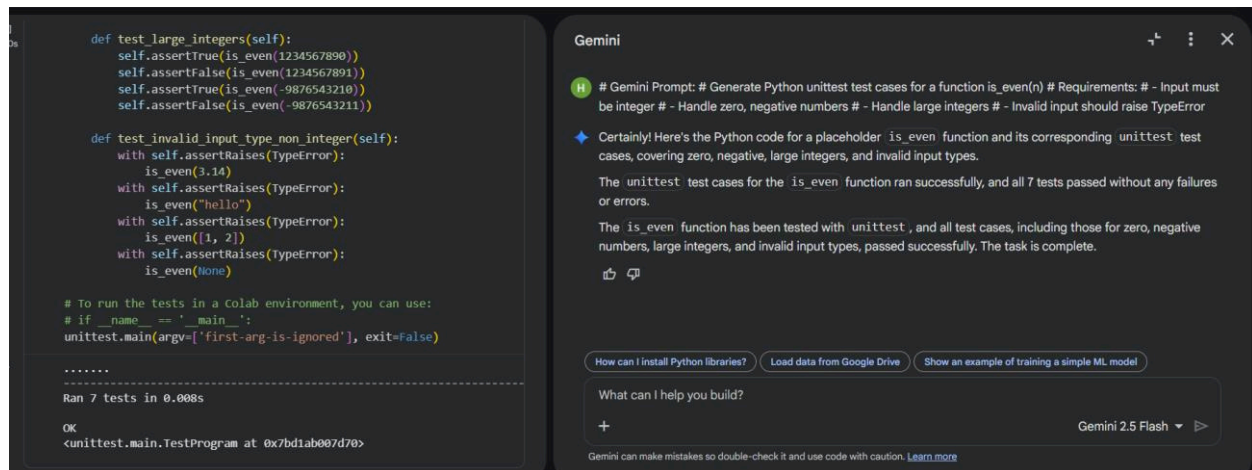
The `is_even` function has been tested with `unittest`, and all test cases, including those for zero, negative numbers, large integers, and invalid input types, passed successfully. The task is complete.



The screenshot shows a code editor on the left and the Gemini chat interface on the right. The code editor contains the following Python code:

```
import unittest  
  
class TestIsEven(unittest.TestCase):  
  
    def test_positive_even_numbers(self):  
        self.assertTrue(is_even(2))  
        self.assertTrue(is_even(100))  
  
    def test_positive_odd_numbers(self):  
        self.assertFalse(is_even(1))  
        self.assertFalse(is_even(99))  
  
    def test_zero(self):  
        self.assertTrue(is_even(0))  
  
    def test_negative_even_numbers(self):  
        self.assertTrue(is_even(-2))  
        self.assertTrue(is_even(-100))  
  
    def test_negative_odd_numbers(self):  
        self.assertFalse(is_even(-1))  
        self.assertFalse(is_even(-99))  
  
    def test_large_integers(self):  
        self.assertTrue(is_even(1234567890))  
        self.assertFalse(is_even(1234567891))  
        self.assertTrue(is_even(-9876543210))
```

The chat interface on the right shows the Gemini prompt and response, which is identical to the one in the previous screenshot.



Explanation: In this task, test cases were generated using Gemini following the Test-Driven Development approach. The `is_even()` function was implemented to satisfy all test conditions, including handling negative numbers, zero, large integers, and raising errors for invalid inputs.

Task 2 – Test-Driven Development for String Case Converter

Ask AI to generate test cases for two functions:

- `to_uppercase(text)`
- `to_lowercase(text)`

Prompt: Generate unittest test cases for `to_uppercase(text)` and `to_lowercase(text)` functions with empty string, mixed-case, and invalid input handling.

```
def to_uppercase(text):
    """
    Converts a given string to uppercase.

    Args:
        text: The string to convert.

    Returns:
        The uppercase version of the string.

    Raises:
        TypeError: If the input is not a string.
    """
    if not isinstance(text, str):
        raise TypeError("Input must be a string.")
    return text.upper()

def to_lowercase(text):
    """
    Converts a given string to lowercase.

    Args:
        text: The string to convert.

    Returns:
        The lowercase version of the string.
    """
```

Gemini

Gemini Prompt: # Generate Python unittest test cases for two functions: # to_uppercase(text) and to_lowercase(text) # Requirements: # - Handle empty strings # - Handle mixed-case input # - Invalid inputs like None or numbers should raise TypeError

Certainly! I'll generate the Python functions to_uppercase and to_lowercase, along with comprehensive unittest test cases covering empty strings, mixed-case inputs, and type validation for invalid inputs.

The unittest test cases for both to_uppercase and to_lowercase functions ran successfully, and all 17 tests passed without any failures or errors. This task is complete.

I have successfully generated and executed the unittest test cases for both the is_even function and the to_uppercase and to_lowercase functions. All tests passed as expected. The requested tasks are complete.

Generate unittest test cases for to_uppercase and to_lowercase

Explain the code for to_uppercase and to_lowercase functions

Add docstrings to the is_even function

What can I help you build?

+

Gemini 2.5 Flash

▶

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

Terminal 10:08 AM Python 3

```
    """
    The lowercase version of the string.

    Raises:
        TypeError: If the input is not a string.
    """
    if not isinstance(text, str):
        raise TypeError("Input must be a string.")
    return text.lower()

Now, let's write the unittest test cases for to_uppercase and to_lowercase functions.
```

import unittest

class TestStringCaseConversion(unittest.TestCase):

 # Test cases for to_uppercase
 def test_uppercase_empty_string(self):
 self.assertEqual(to_uppercase(""), "")

 def test_uppercase_mixed_case(self):
 self.assertEqual(to_uppercase("Hello World"), "HELLO WORLD")

 def test_uppercase_all_lowercase(self):
 self.assertEqual(to_uppercase("python programming"), "PYTHON PROGRAMMING")

 def test_uppercase_all_uppercase(self):
 self.assertEqual(to_uppercase("ALREADY UPPERCASE"), "ALREADY UPPERCASE")

 def test_uppercase_invalid_input_type(self):
 with self.assertRaises(TypeError):
 to_uppercase(None)
 with self.assertRaises(TypeError):
 to_uppercase(123)
 with self.assertRaises(TypeError):
 to_uppercase(['a', 'b'])

 # Test cases for to_lowercase
 def test_lowercase_empty_string(self):
 self.assertEqual(to_lowercase(""), "")

 def test_lowercase_mixed_case(self):
 self.assertEqual(to_lowercase("Hello world"), "hello world")

 def test_lowercase_all_uppercase(self):
 self.assertEqual(to_lowercase("PYTHON PROGRAMMING"), "python programming")

 def test_lowercase_all_lowercase(self):
 self.assertEqual(to_lowercase("already lowercase"), "already lowercase")

 def test_lowercase_invalid_input_type(self):
 with self.assertRaises(TypeError):

Gemini

Gemini Prompt: # Generate Python unittest test cases for two functions: # to_uppercase(text) and to_lowercase(text) # Requirements: # - Handle empty strings # - Handle mixed-case input # - Invalid inputs like None or numbers should raise TypeError

Certainly! I'll generate the Python functions to_uppercase and to_lowercase, along with comprehensive unittest test cases covering empty strings, mixed-case inputs, and type validation for invalid inputs.

The unittest test cases for both to_uppercase and to_lowercase functions ran successfully, and all 17 tests passed without any failures or errors. This task is complete.

I have successfully generated and executed the unittest test cases for both the is_even function and the to_uppercase and to_lowercase functions. All tests passed as expected. The requested tasks are complete.

Generate unittest test cases for to_uppercase and to_lowercase

Explain the code for to_uppercase and to_lowercase functions

Add docstrings to the is_even function

What can I help you build?

+

Gemini 2.5 Flash

▶

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

Terminal 10:08 AM Python 3

```
self.assertEqual(to_uppercase("python programming"), "PYTHON PROGRAMMING")

def test_uppercase_all_uppercase(self):
    self.assertEqual(to_uppercase("ALREADY UPPERCASE"), "ALREADY UPPERCASE")

def test_uppercase_invalid_input_type(self):
    with self.assertRaises(TypeError):
        to_uppercase(None)
    with self.assertRaises(TypeError):
        to_uppercase(123)
    with self.assertRaises(TypeError):
        to_uppercase(['a', 'b'])

# Test cases for to_lowercase
def test_lowercase_empty_string(self):
    self.assertEqual(to_lowercase(""), "")

def test_lowercase_mixed_case(self):
    self.assertEqual(to_lowercase("Hello world"), "hello world")

def test_lowercase_all_uppercase(self):
    self.assertEqual(to_lowercase("PYTHON PROGRAMMING"), "python programming")

def test_lowercase_all_lowercase(self):
    self.assertEqual(to_lowercase("already lowercase"), "already lowercase")

def test_lowercase_invalid_input_type(self):
    with self.assertRaises(TypeError):
```

Gemini

Gemini Prompt: # Generate Python unittest test cases for two functions: # to_uppercase(text) and to_lowercase(text) # Requirements: # - Handle empty strings # - Handle mixed-case input # - Invalid inputs like None or numbers should raise TypeError

Certainly! I'll generate the Python functions to_uppercase and to_lowercase, along with comprehensive unittest test cases covering empty strings, mixed-case inputs, and type validation for invalid inputs.

The unittest test cases for both to_uppercase and to_lowercase functions ran successfully, and all 17 tests passed without any failures or errors. This task is complete.

I have successfully generated and executed the unittest test cases for both the is_even function and the to_uppercase and to_lowercase functions. All tests passed as expected. The requested tasks are complete.

Generate unittest test cases for to_uppercase and to_lowercase

Explain the code for to_uppercase and to_lowercase functions

Add docstrings to the is_even function

What can I help you build?

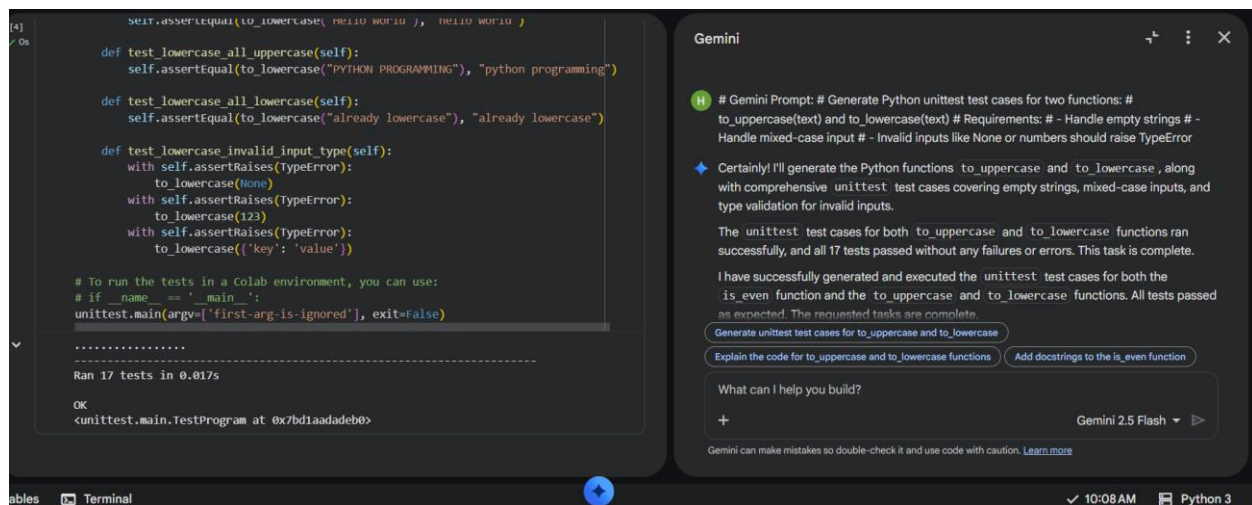
+

Gemini 2.5 Flash

▶

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

Terminal 10:08 AM Python 3

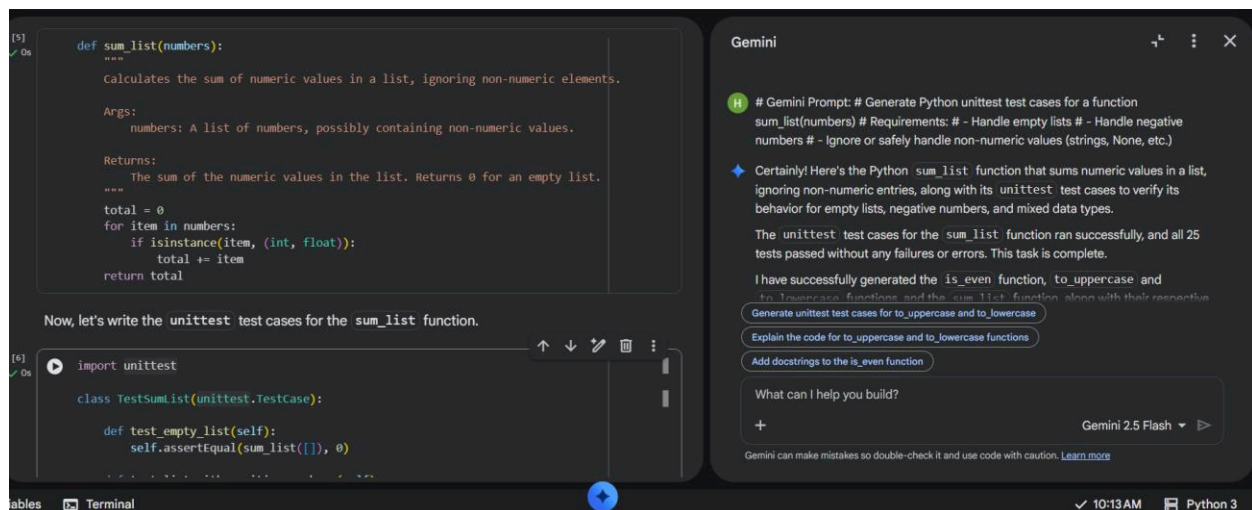


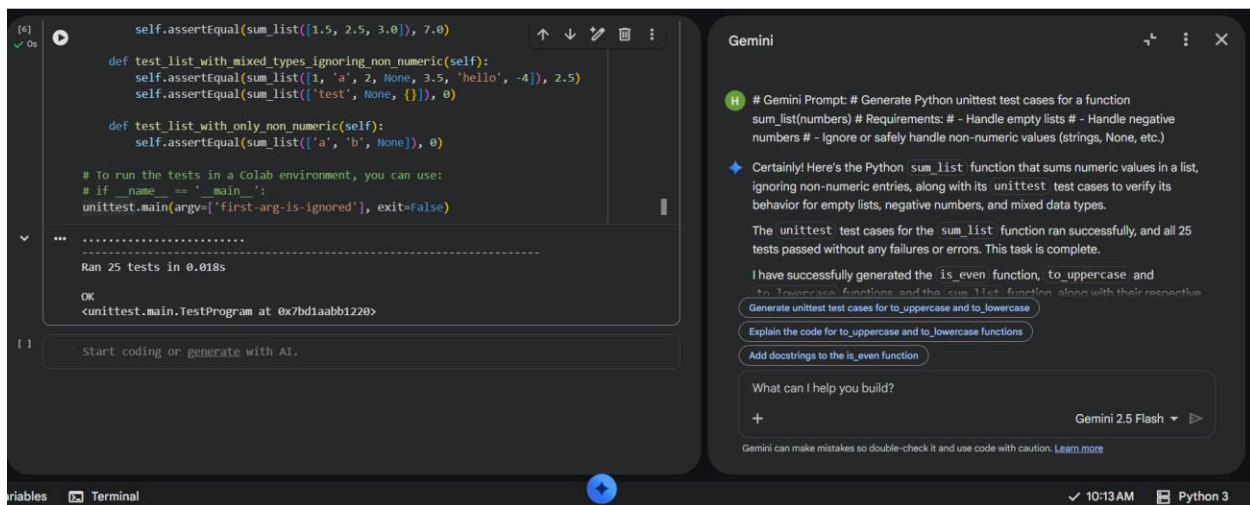
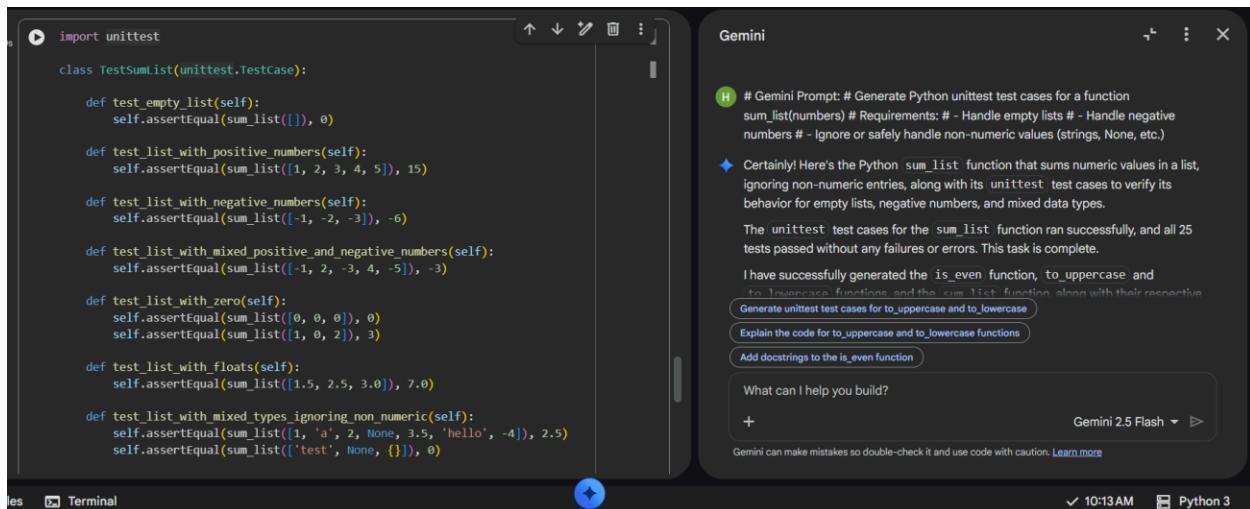
Explanation: Using Gemini, test cases were generated before implementation following the Test-Driven Development approach. The functions were implemented to correctly convert string cases and safely handle invalid inputs by raising Type Error.

Task 3 – Test-Driven Development for List Sum Calculator

Use AI to generate test cases for a function `sum_list(numbers)` that calculates the sum of list elements.

Prompt: Generate unittest test cases for a function `sum_list(numbers)` that handles empty lists, negative numbers, and ignores non-numeric values.





Explanation: In this task, Gemini was used to generate test cases following the Test-Driven Development approach. The sum_list() function was implemented to correctly compute the sum of numeric values while safely ignoring non-numeric elements and handling empty lists.

Task 4 – Test Cases for Student Result Class

Generate test cases for a StudentResult class with the following methods:

- add_marks(mark)
- calculate_average()
- get_result()

Prompt: Generate unittest test cases for a Student Result class with mark validation, average calculation, and pass/fail decision.

```
[7] 0s
class StudentResult:
    """
    A class to manage a student's marks and calculate their result.
    """
    def __init__(self):
        self.marks = []

    def add_marks(self, mark):
        """
        Adds a mark to the student's record.

        Args:
            mark (int): The mark to add.

        Raises:
            ValueError: If the mark is not between 0 and 100 (inclusive).
        """
        if not isinstance(mark, int) or not (0 <= mark <= 100):
            raise ValueError("Mark must be an integer between 0 and 100.")
        self.marks.append(mark)

    def calculate_average(self):
        """
        Calculates the average of all recorded marks.

        Returns:
            float: The average mark. Returns 0 if no marks have been added.
        """
```

Gemini

Gemini Prompt: # Generate Python unittest test cases for a StudentResult class #
Methods: # - add_marks(mark) # - calculate_average() # - get_result() #
Requirements: # - Marks must be between 0 and 100 # - Average >= 40 should return "Pass", else "Fail" # - Invalid marks (negative or >100) should raise ValueError

Certainly! Here's the Python code for the StudentResult class, implementing the add_marks, calculate_average, and get_result methods. Following that, I've provided a comprehensive set of unittest test cases to verify all requirements, including handling valid and invalid marks, calculating averages, and determining pass/fail statuses.

The unittest test cases for the StudentResult class ran successfully, and all 36 tests passed without any failures or errors. This task is complete.

Instantiate the StudentResult class and add some marks

Explain how to use the StudentResult class methods

Add more test cases for edge scenarios for StudentResult

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

```
[7] 0s
    if not self.marks:
        return 0.0
    return sum(self.marks) / len(self.marks)

    def get_result(self):
        """
        Determines if the student passed or failed based on their average mark.

        Returns:
            str: "Pass" if the average mark is 40 or greater, "Fail" otherwise.
        """
        average = self.calculate_average()
        return "Pass" if average >= 40 else "Fail"

Now, let's write the unittest test cases for the StudentResult class.

[8] 0s
import unittest

class TestStudentResult(unittest.TestCase):

    def setUp(self):
        """Set up a new StudentResult instance before each test method."""
        self.student = StudentResult()

    def test_add_valid_marks(self):
```

Gemini

Gemini Prompt: # Generate Python unittest test cases for a StudentResult class #
Methods: # - add_marks(mark) # - calculate_average() # - get_result() #
Requirements: # - Marks must be between 0 and 100 # - Average >= 40 should return "Pass", else "Fail" # - Invalid marks (negative or >100) should raise ValueError

Certainly! Here's the Python code for the StudentResult class, implementing the add_marks, calculate_average, and get_result methods. Following that, I've provided a comprehensive set of unittest test cases to verify all requirements, including handling valid and invalid marks, calculating averages, and determining pass/fail statuses.

The unittest test cases for the StudentResult class ran successfully, and all 36 tests passed without any failures or errors. This task is complete.

Instantiate the StudentResult class and add some marks

Explain how to use the StudentResult class methods

Add more test cases for edge scenarios for StudentResult

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

```
[8] 0s
        self.student = StudentResult()

    def test_add_valid_marks(self):
        self.student.add_marks(75)
        self.student.add_marks(80)
        self.assertEqual(self.student.marks, [75, 80])

    def test_add_mark_below_zero_raises_error(self):
        with self.assertRaises(ValueError):
            self.student.add_marks(-5)

    def test_add_mark_above_hundred_raises_error(self):
        with self.assertRaises(ValueError):
            self.student.add_marks(101)

    def test_add_non_integer_mark_raises_error(self):
        with self.assertRaises(ValueError):
            self.student.add_marks(75.5)
        with self.assertRaises(ValueError):
            self.student.add_marks("seventy")

    def test_calculate_average_empty_list(self):
        self.assertEqual(self.student.calculate_average(), 0.0)

    def test_calculate_average_with_marks(self):
        self.student.add_marks(70)
        self.student.add_marks(80)
        self.student.add_marks(90)
```

Gemini

Gemini Prompt: # Generate Python unittest test cases for a StudentResult class #
Methods: # - add_marks(mark) # - calculate_average() # - get_result() #
Requirements: # - Marks must be between 0 and 100 # - Average >= 40 should return "Pass", else "Fail" # - Invalid marks (negative or >100) should raise ValueError

Certainly! Here's the Python code for the StudentResult class, implementing the add_marks, calculate_average, and get_result methods. Following that, I've provided a comprehensive set of unittest test cases to verify all requirements, including handling valid and invalid marks, calculating averages, and determining pass/fail statuses.

The unittest test cases for the StudentResult class ran successfully, and all 36 tests passed without any failures or errors. This task is complete.

Instantiate the StudentResult class and add some marks

Explain how to use the StudentResult class methods

Add more test cases for edge scenarios for StudentResult

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

```
self.student.add_marks(10)
self.student.add_marks(80)
self.student.add_marks(90)
self.assertEqual(self.student.calculate_average(), 80.0)

def test_get_result_pass(self):
    self.student.add_marks(50)
    self.student.add_marks(60)
    self.student.add_marks(70)
    self.assertEqual(self.student.get_result(), "Pass")

def test_get_result_fail(self):
    self.student.add_marks(20)
    self.student.add_marks(30)
    self.student.add_marks(40)
    self.assertEqual(self.student.get_result(), "Fail")

def test_get_result_borderline_pass(self):
    self.student.add_marks(40)
    self.assertEqual(self.student.get_result(), "Pass")

def test_get_result_borderline_fail(self):
    self.student.add_marks(39)
    self.assertEqual(self.student.get_result(), "Fail")

def test_get_result_empty_marks_list_fail(self):
    self.assertEqual(self.student.get_result(), "Fail") # Average is 0.0
```

Gemini

... provided a comprehensive set of unittest test cases to verify all requirements, including handling valid and invalid marks, calculating averages, and determining pass/fail statuses.

The unittest test cases for the StudentResult class ran successfully, and all 36 tests passed without any failures or errors. This task is complete.

I have successfully generated and executed the unittest test cases for all the requested functions and classes: is_even, to_uppercase, to_lowercase, sum_list, and StudentResult. All tests ran successfully and passed without any issues. The overall task is now complete.

Instantiate the StudentResult class and add some marks

Explain how to use the StudentResult class methods

Add more test cases for edge scenarios for StudentResult

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

```
self.student.add_marks(30)
self.student.add_marks(40)
self.assertEqual(self.student.get_result(), "Fail")

def test_get_result_borderline_pass(self):
    self.student.add_marks(40)
    self.assertEqual(self.student.get_result(), "Pass")

def test_get_result_borderline_fail(self):
    self.student.add_marks(39)
    self.assertEqual(self.student.get_result(), "Fail")

def test_get_result_empty_marks_list_fail(self):
    self.assertEqual(self.student.get_result(), "Fail") # Average is 0.0

# To run the tests in a Colab environment, you can use:
# if __name__ == '__main__':
#     unittest.main(argv=['first-arg-is-ignored'], exit=False)
```

Ran 36 tests in 0.029s

OK

<unittest.main.TestProgram at 0x7bd1aad0c410>

Gemini

... provided a comprehensive set of unittest test cases to verify all requirements, including handling valid and invalid marks, calculating averages, and determining pass/fail statuses.

The unittest test cases for the StudentResult class ran successfully, and all 36 tests passed without any failures or errors. This task is complete.

I have successfully generated and executed the unittest test cases for all the requested functions and classes: is_even, to_uppercase, to_lowercase, sum_list, and StudentResult. All tests ran successfully and passed without any issues. The overall task is now complete.

Instantiate the StudentResult class and add some marks

Explain how to use the StudentResult class methods

Add more test cases for edge scenarios for StudentResult

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

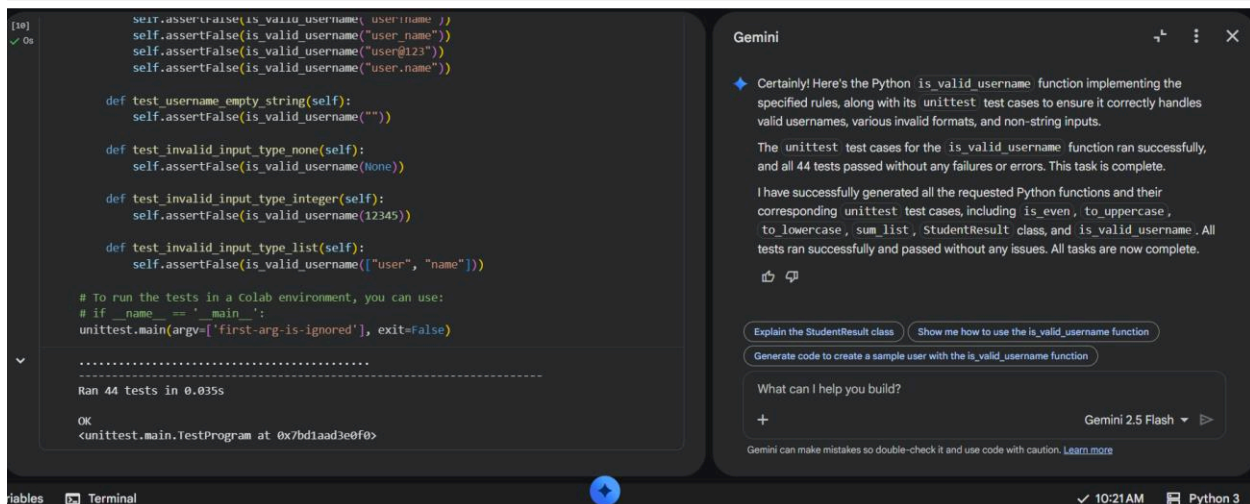
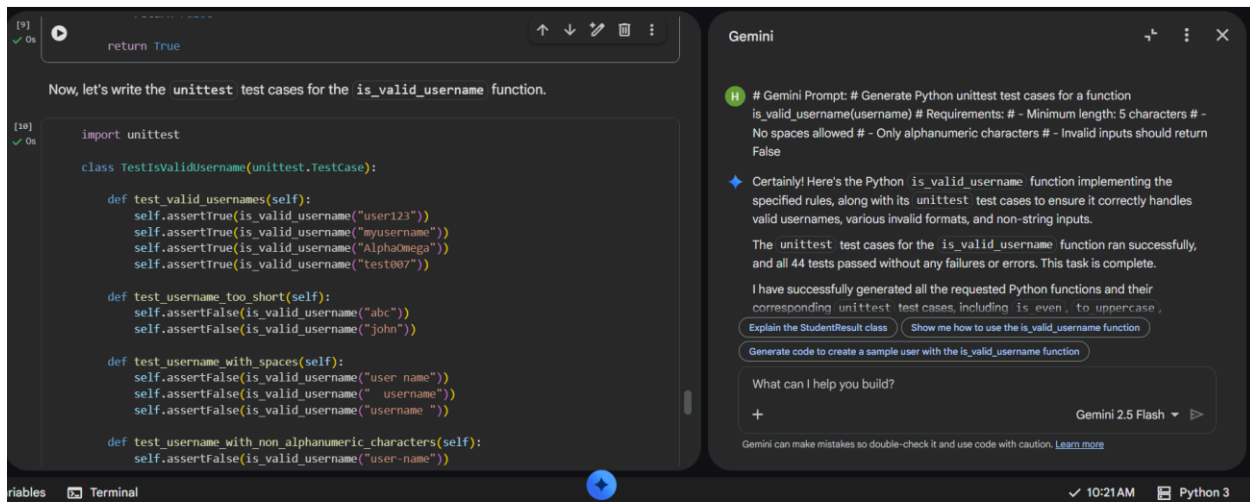
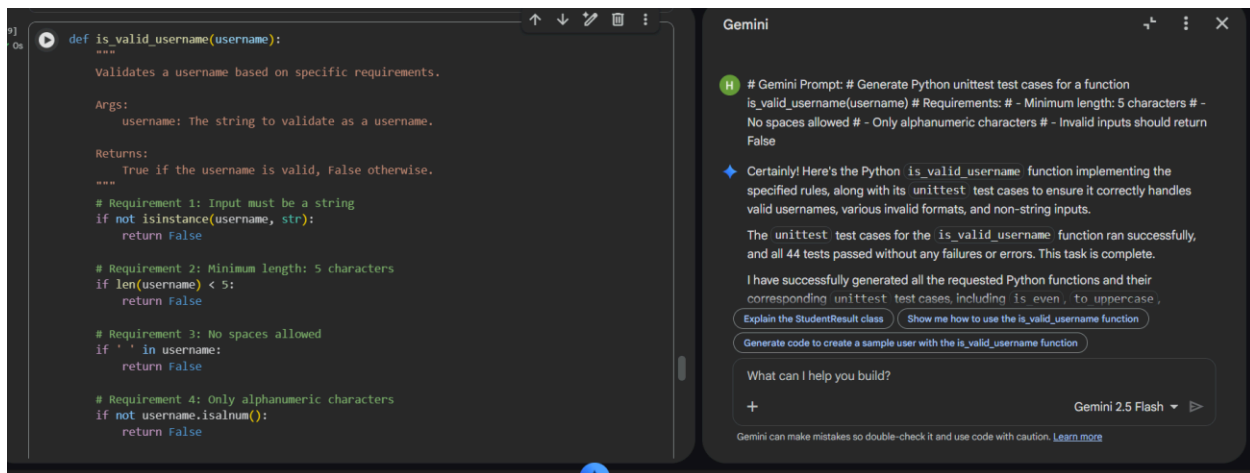
Explanation: Using Gemini, test cases were generated for the StudentResult class following Test-Driven Development. The class was implemented to validate marks, calculate average scores, and determine pass or fail status based on defined conditions.

Task 5 – Test-Driven Development for Username Validator

Requirements:

- Minimum length: 5 characters
- No spaces allowed
- Only alphanumeric characters

Prompt: Generate unittest test cases for a username validation function with length, space, and character constraints.



Explanation: In this task, Gemini was used to generate test cases for validating usernames using the Test-Driven Development approach. The `is_valid_username()` function was implemented to enforce length, spacing, and alphanumeric constraints as per the test cases.